

DOCUMENTAÇÃO DE ARQUITETURA

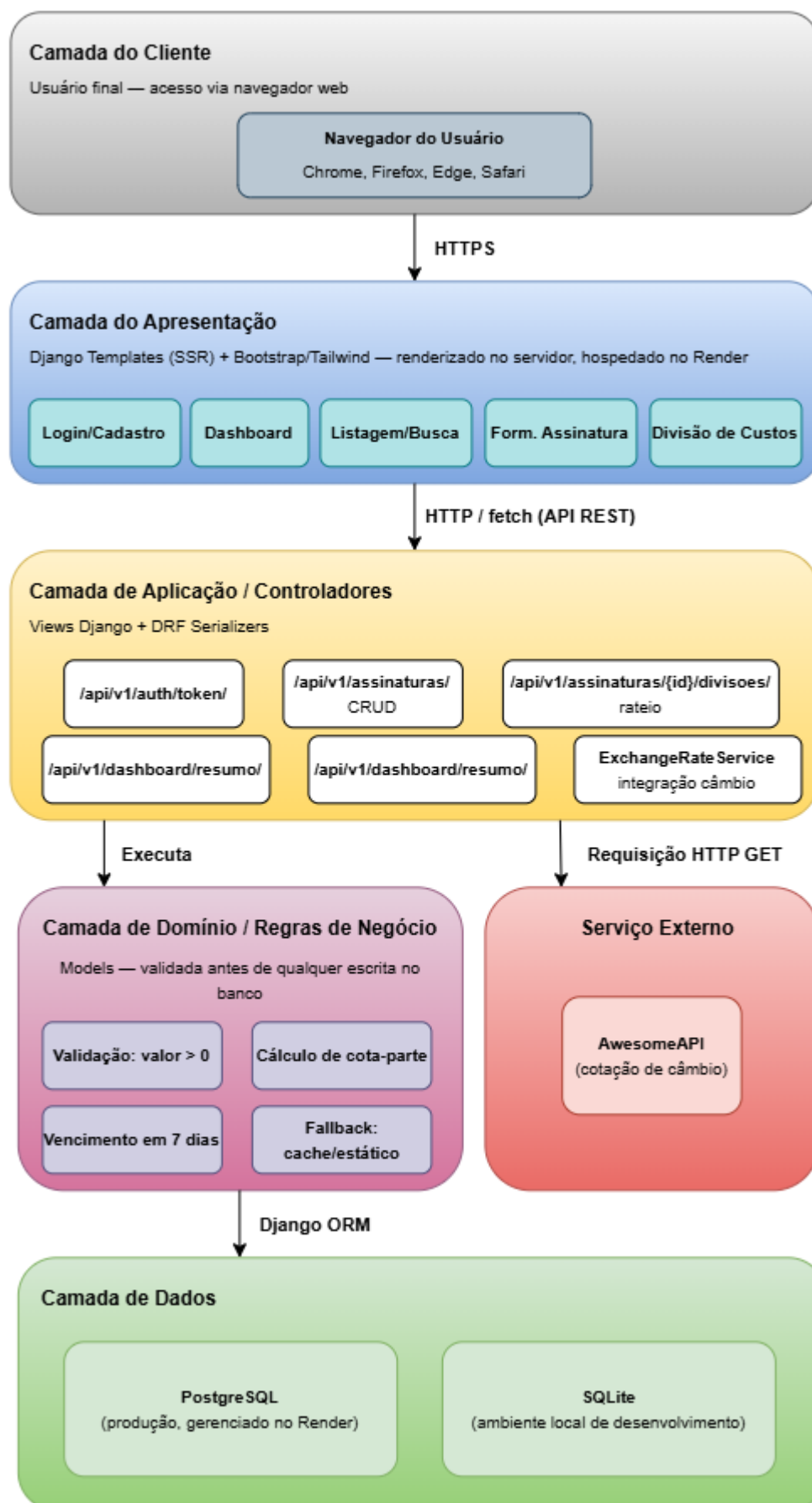
Arquitetura do sistema

SubTracker

1. Modelo de Arquitetura e Decisões Técnicas

- **Frontend:** utilização do padrão Django Templates (MVT) com renderização no lado do servidor (Server-Side Rendering - SSR), complementado com HTML5, CSS3, JavaScript e Bootstrap/Tailwind para dinamismo na interface (ex: requisições assíncronas via fetch para a API REST e dinamismo em formulários).
- **Hospedagem e Infraestrutura:** deploy da aplicação web e da API REST na plataforma Render (ambiente de execução web hospedado via HTTPS), interligada a uma instância de base de dados relacional PostgreSQL gerida no próprio provedor.
- **Padrão de Camadas:** organização baseada no padrão arquitetural MVT (Model-View-Template) do Django, segregando responsabilidades em Mapeamento/Negócio, Processamento/Controladores e Apresentação.

2. Diagrama de Componentes



3. Descrição das Camadas e Responsabilidades

Camada de Apresentação (Templates e Assets)

- Responsável pela interface gráfica e interação com o utilizador.
- Renderiza as páginas HTML do Dashboard, formulários de assinaturas, relatórios e gestão de utilizadores.
- Executa scripts JavaScript no navegador para chamadas assíncronas aos endpoints da API REST do projeto.

Camada de Aplicação / Controladores (Views, DRF Serializers e Services)

- **Views & DRF Serializers:** processam as requisições HTTP, realizam a validação de formulários/payloads JSON e controlam o fluxo de navegação e respostas HTTP (200 OK, 201 Created, 400 Bad Request, etc.).
- **Service Layer (ExchangeRateService):** módulo dedicado para isolar a integração com serviços externos. Responsável por disparar requisições HTTP para a AwesomeAPI, aplicar o tratamento de timeouts e gerir a cotação de fallback em caso de falha de conexão.

Camada de Domínio e Modelo (Models & Business Logic)

- Define as entidades do sistema (User, Subscription, Category, SplitGroup/PersonSplit) e as regras de negócio.
- Executa os cálculos financeiros (conversão de moeda, soma de gastos mensais/anuais e divisão proporcional de custos por participante).

Camada de Persistência (ORM & Banco de Dados)

- Intermediada pelo Django ORM para abstração das consultas SQL.
- Em ambiente local/desenvolvimento utiliza SQLite; em ambiente de produção/hospedagem utiliza PostgreSQL.

4. Fluxo de Dados e Integração Externa (Câmbio)

- O utilizador acede ao Dashboard ou à visualização de uma assinatura em moeda estrangeira.
- A camada de visão (View) aciona o serviço ExchangeRateService no backend.
- O ExchangeRateService envia uma requisição HTTP GET para a AwesomeAPI obtendo a cotação em tempo real.
- **Tratamento de Exceção/Resiliência:** se a API externa responder com erro ou exceder o tempo limite (timeout), o serviço captura a exceção e retorna a taxa de câmbio pré-configurada (fallback) armazenada na aplicação, garantindo a exibição sem interromper a execução.
- A View utiliza os dados da model para aplicar a conversão e envia os valores consolidados para o Template ou resposta.

